TimeWeave

Clash-free college timetables, generated in seconds

Hackathon · Day 1 progress | Wednesday 21 October 2026 | Team: Dilnaz Grewal

Summary

On Day 1 we built a **working end-to-end prototype**. It generates the weekly timetable for a whole college (5 engineering departments plus Applied Sciences) using Google OR-Tools **CP-SAT**. A separate **independent validator** then proves that the result has no clashes. The app shows the result by section, teacher or room on desktop and phone. It also includes a free-room finder, teacher-load and room-usage dashboards, a read-only AI assistant, and one-page PDF printouts.

✓ 0
clashes, confirmed by an independent validator

394 / 394
weekly sessions scheduled (20 sections, 39 teachers, 38 rooms)

~33 s
to generate the whole college timetable

1.2 → 0.07
student idle periods per day (unoptimised → optimised)

5 → 3
days with only one lecture (unoptimised → optimised)

12 / 12
automated tests passing

The problem

- Teachers build the semester timetable **by hand**. It takes weeks and still ends up with clashes, uneven teaching loads and empty rooms.
- Students wait for hours between lectures (for example, a class at 12:30 and the next at 3:30).
- The timetable is fragile: one change triggers a chain of others, so problems go unfixed.
- Nothing helps with daily changes, such as an absent teacher or a student looking for an empty room.

Our approach

- **Constraint solving, not trial and error:** hard rules can never be broken, and soft goals (fewer gaps, balanced days) are optimised.
- **Trust through proof:** a separate validator re-checks every timetable, and tests inject clashes to prove that it catches them.
- **Student-friendly days:** the solver minimises idle gaps and gives each section a flexible lunch hour.
- **Simple for every role:** a timetable office, department coordinators, teachers and students each see what they need, on a phone too.

Architecture

How a timetable is made

1 · Input

Seeded college data today. An **Excel/CSV template upload** comes next (Rooms, Teachers, Sections, Assignments, Settings).

▼

2 · Pre-checks

Plain-English errors before solving, e.g. *"Prof. X is assigned 32 hours but can teach at most 28 with their availability."*

▼

3 · CP-SAT solver (Google OR-Tools)

Hard rules (never broken):

- No double-booking of a teacher, a student batch or a room.
- Teacher availability is respected (part-time and visiting staff).
- Labs are 2-hour blocks in a lab of the right type with enough seats.
- Flexible lunch: everyone gets a free hour between 11:30 and 2:30.
- Daily limits apply, and a subject is taught at most once a day.

Soft goals (minimised): idle gaps between lectures, days with a single lecture, gaps in teachers' days, and 4 or more periods in a row.

▼

4 · Independent validator

Separate code with no shared logic re-checks every result: teachers, students, rooms, room type and capacity, availability, lunch, daily limits, and that every session is placed. It produces the **"✓ 0 clashes"** badge.

▼

5 · Saved timetable

The weekly base timetable (cached, so the app starts instantly). Date-specific overrides for absences and bookings come next.

▼

6 · Outputs

Grids by section, teacher or room · free-room finder · teacher-load and room-usage dashboards · AI assistant (read-only) · PDF printouts.

Technology

Layer	Choice and reason
Solver	**Google OR-Tools CP-SAT**: proves hard rules hold and optimises soft goals. Chosen over genetic algorithms, which cannot guarantee feasibility or explain failures.
Backend	**Python + FastAPI**: a single language for the solver, the API and the AI assistant.
Frontend	**One HTML page** with Tailwind CSS and Alpine.js: no build step, phone-first, with print-to-PDF built in.
AI assistant	**Claude (Anthropic API)** with read-only tools over the live timetable. It has no ability to change data.
Storage	In-memory plus a JSON cache today. SQLite comes next, then Postgres (Neon free tier) when deployed.
Quality	pytest (12 tests), git history with a commit at every milestone.

Design rules we hold to

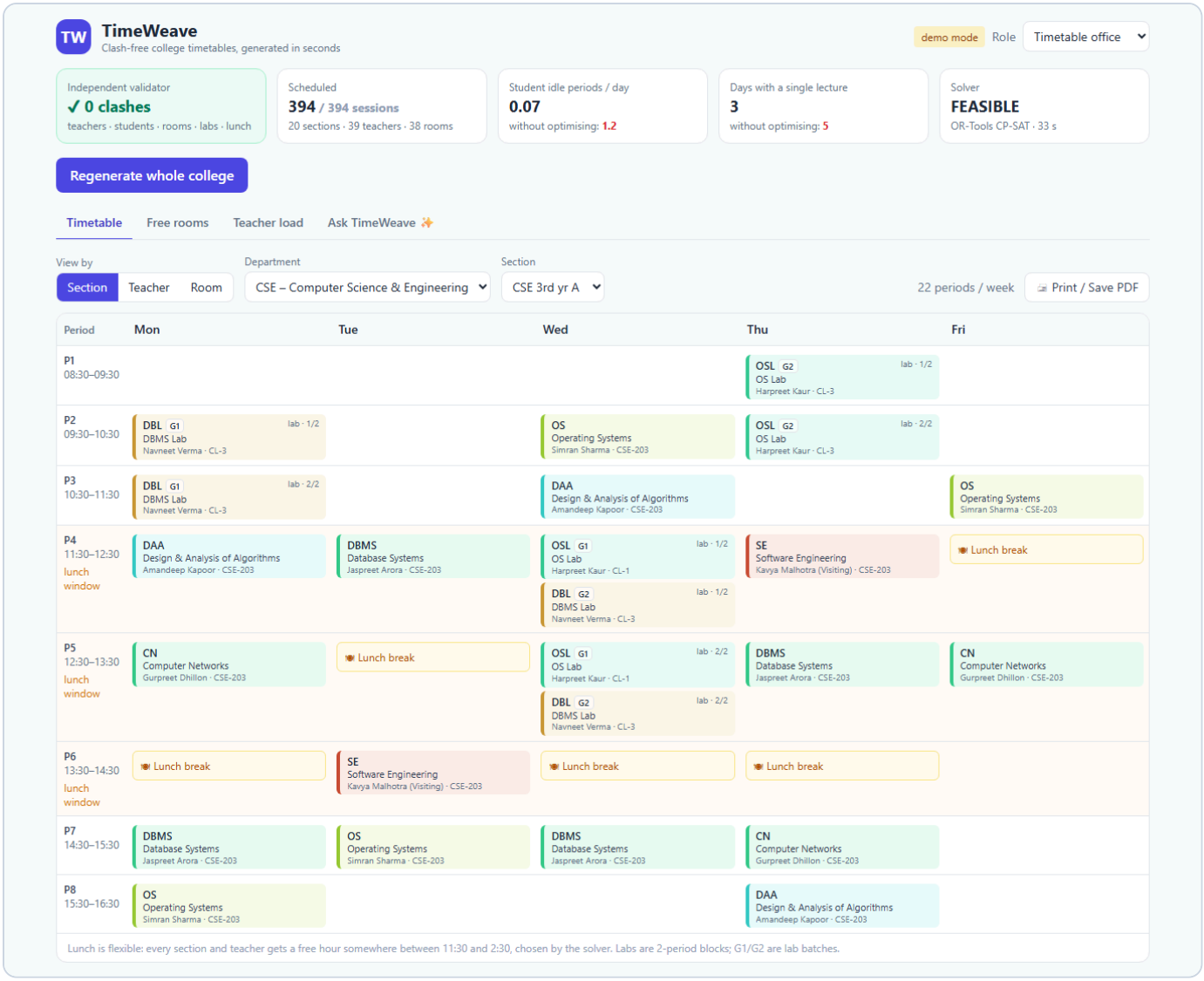
- The validator never shares code with the solver, so the solver never grades its own work.
- Every future change (editor, leave request, chatbot) goes through one change engine: *propose → validate → preview → apply*.
- The chatbot can only read or propose. A person always confirms.
- Permissions are enforced on the server, not in the UI or in the AI prompt.

Features completed today (version 0.1)

#	Feature	What it does	Status
1	Realistic college data	CSE, IT, ECE, ME, CE and Applied Sciences; 20 sections in 2nd and 3rd year; 39 teachers; 38 rooms; 394 weekly sessions. Includes lab batches G1/G2, computer labs shared by CSE and IT, teachers shared across departments, and a visiting teacher available only Tue/Thu.	✅ **Done**
2	Whole-college auto-generation	CP-SAT solver with all the hard rules and soft goals above. One click generates every department together, in about 33 seconds.	✅ **Done**
3	Flexible lunch	Lunch is not fixed at 12:30. Every section and teacher gets a free hour somewhere between 11:30 and 2:30, chosen by the solver, and it is not counted as idle time.	✅ **Done**
4	Student-friendly days	Idle gaps between lectures and days with a single lecture are minimised: idle periods drop from 1.2 to 0.07 per day, and single-lecture days from 5 to 3.	✅ **Done**
5	Pre-checks with plain-English errors	Impossible inputs (an overloaded teacher, too few labs) are explained before solving.	✅ **Done**
6	Independent validator	Re-checks every timetable and shows "✓ 0 clashes", or lists each problem.	✅ **Done**
7	Timetable views	Weekly grid by section, teacher or room, filtered by department. Labs are shown as 2-period blocks with their batch, and each section's lunch break is marked.	✅ **Done**
8	Phone layout	On a phone, the timetable shows one day at a time with day tabs.	✅ **Done**
9	Free-room finder	Lists the rooms that are empty in any period, with a "Right now" button and how long each room stays free.	✅ **Done**
10	Dashboards	Teaching load per teacher; classroom and lab utilisation.	✅ **Done**
11	AI assistant: "Ask TimeWeave"	Answers questions such as "When is Prof. X free on Friday?" using 4 read-only tools: search, schedule, free rooms and teacher free slots. It answers only from live data and cannot change anything. Tested with the live API.	✅ **Done**
12	Print / Save as PDF	A one-page A4 landscape timetable for any section, teacher or room, with a verification stamp. Ready for notice boards.	✅ **Done**
13	Roles	A switcher for the timetable office, coordinator, teacher and student views. Real login comes on Day 3.	🟡 **Demo mode**
14	Automated tests	12 tests: the solver output validates clean, and each kind of clash injected on purpose is caught.	✅ **Done**
15	Documentation	`CLAUDE.md` (engineering rules), `PLAN.md` (end-to-end plan), `docs/DESIGN.md` (detailed design), `README.md` (how to run, demo script).	✅ **Done**

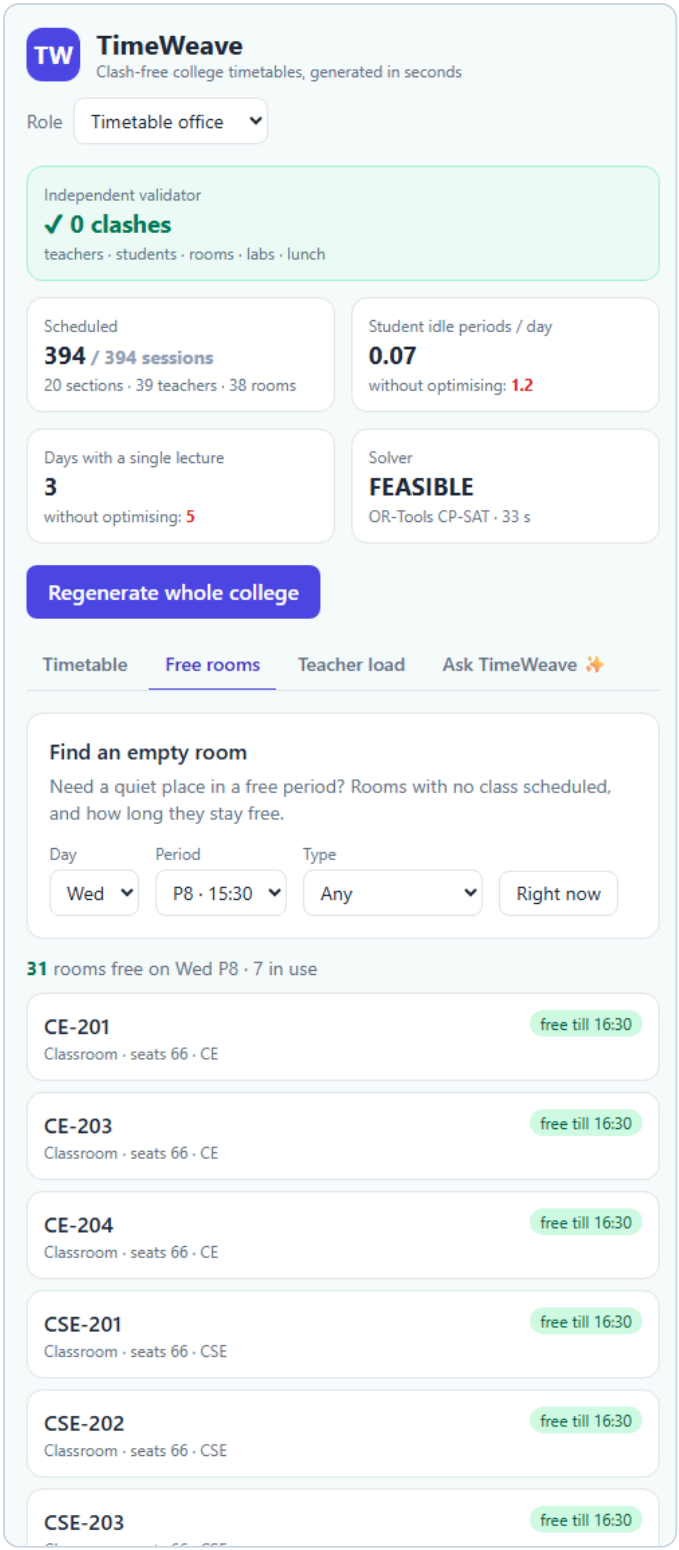
Deliberate shortcuts for Day 1: data lives in memory plus a JSON file rather than a database, and roles are a demo switcher rather than real login. Both are scheduled in the roadmap (page 7).

Screenshots: the generated timetable

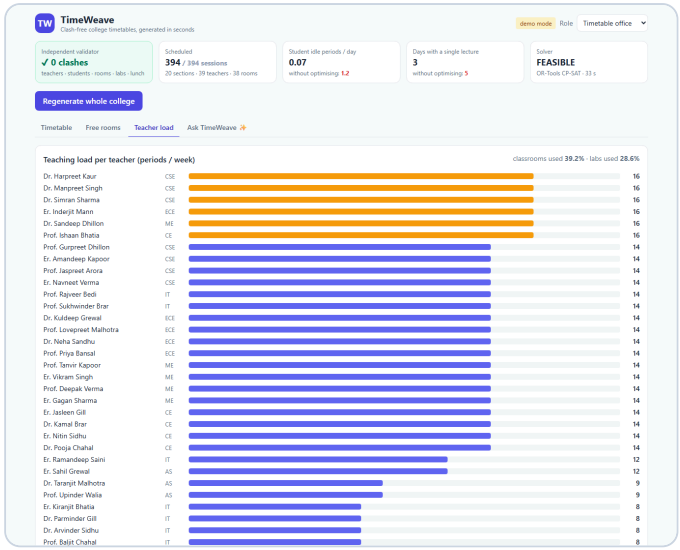


Section view: CSE 3rd yr A. The top cards show the validator result (✓ 0 clashes), 394/394 sessions scheduled, and idle periods and single-lecture days compared with an unoptimised solution. Labs (OSL, DBL) run as 2-hour blocks for batches G1 and G2 in the shared computer labs. The 🍽 lunch break moves within 11:30–2:30 from day to day. The "Print / Save PDF" button produces a one-page A4 version.

Screenshots: phone, dashboards and assistant



Free-room finder on a phone. Pick a day and period (or press "Right now") to see empty rooms and how long they stay free.



Teacher load dashboard. Periods per week for every teacher, plus classroom and lab utilisation.

Ask TimeWeave: real exchange from the live test

> What does CSE-3A have on Monday?

Here's CSE-3A's Monday. All classes are in room CSE-203:
· 09:30–10:30 (P2): Database Systems with Prof. Jaspreet Arora
· 10:30–11:30 (P3): Operating Systems with Dr. Simran Sharma
· 11:30–12:30 (P4): Computer Networks with Prof. Gurpreet Dhillon

checked: get_schedule

The assistant looked the answer up with a tool rather than guessing. This exchange was recorded on an earlier version of the timetable, before it was regenerated.

Verification and testing

How we show the timetable never contains a clash:

Check	How	Result
Solver output is clean	The seeded college is solved, and the independent validator reports zero issues.	✅ **Pass**
Every session is placed exactly once	Count of placed sessions = required sessions (394).	✅ **Pass**
Student clash is caught	Move a class onto another class of the same section; the validator must flag it.	✅ **Pass**
Room clash is caught	Put two labs in the same lab at the same time.	✅ **Pass**
Wrong room type is caught	Put a lab in a lecture theatre.	✅ **Pass**
Unavailable teacher is caught	Move the visiting teacher's class to Monday.	✅ **Pass**
Missing lunch is caught	Fill all of 11:30–2:30 for a section.	✅ **Pass**
Everyone gets lunch	No section or teacher in the generated timetable lacks a lunch hour.	✅ **Pass**
Block past end of day is caught	Start a 2-hour lab in the last period.	✅ **Pass**
Missing session is caught	Remove one session.	✅ **Pass**
Overloaded teacher is explained	Pre-check names the teacher and their hours.	✅ **Pass**
Unknown data is rejected	An entry for a lesson that doesn't exist is flagged.	✅ **Pass**

Other manual checks: free-room results match the room grids; the PDF fits one A4 page (checked by printing with Edge); the AI assistant's answer matches the grid; the app works at phone width.

Development log (git commits, 21 Oct 2026)

Time	Commit	Change
14:30	`6a7fdbf`	CLAUDE.md, plan and project scaffold
14:39	`e6b8426`	Seed college, CP-SAT solver, independent validator, API and UI
14:44	`6ebb864`	Validator mutation tests and solver golden test
14:46	`8607fbe`	Read-only Claude timetable assistant with 4 tools
14:47	`b3f27f5`	README with run instructions and demo script; cached demo solution
15:03	`5a3cd66`	Initial models and seed data for college timetable system
15:10	`cb4c339`	Flexible lunch: a free hour anywhere in 11:30–14:30 per section and teacher
15:11	`6f61e73`	Plain-text assistant answers for clean chat display
15:31	`cc17526`	Print / Save as PDF (one-page A4 timetable)
15:32	`9790d63`	End-to-end PLAN.md; detailed design moved to docs/DESIGN.md
15:32	`e06b01c`	Demo script updated with current numbers

What's next: roadmap to submission

Version	Features	Done when...
v0.1 · Day 1 ✅ **Complete**	Everything on page 3.	Whole college generated and verified clash-free, and the demo runs.
v0.2 · Day 2 Real data and editing	• Excel/CSV template upload with row-level errors. • Check an **existing hand-made timetable** for clashes and gaps. • Copy last semester's setup. • Settings for days, periods and the lunch window. • Database storage. • Drag-and-drop editor with live clash warnings. • Draft and published versions, rollback, and an audit log. • Excel export.	A coordinator uploads Excel, fixes errors, generates, edits safely and publishes.
v0.3 · Day 3 Daily operations	• Teacher leave → **ranked substitutes** → coordinator confirms. • Swap or cancel as alternatives. • Student "Today" view with live changes. • Free rooms that account for cancellations and bookings. • Google login with role-based permissions. • Notifications and a calendar (.ics) feed.	An absence is handled end to end, and students see it for that day only.
v1.0 · Day 4 Submission	• **Minimal-change repair** with a preview of exactly what will change. • The chatbot proposes changes and a person confirms them. • Explanations when no timetable is possible. • A QR code page for each room door. • Deployment on free hosting, and final polish.	Every headline feature can be demoed live from the deployed link on a phone.

Planned input and output formats

Input: one Excel workbook (each sheet also accepted as CSV)

- **Settings**: working days, period times, lunch window
- **Rooms**: id, type, capacity, owning department
- **Teachers**: name, department, type, availability
- **Sections**: department, year, size, home room, G1/G2 split
- **Assignments**: teacher + subject + section, theory/lab, hours/week

Outputs

- Live web link and QR code: always current ✅ **link**
- One-page PDF per section, teacher or room ✅
- Excel export in the same format as the input **Day 2**
- Calendar feed (.ics) for phones **Day 3**

How to run: `pip install -r requirements.txt`, then `uvicorn app.main:app --port 8000`, then open `http://localhost:8000`. The full plan is in `PLAN.md`.